Literature Survey & Research Gap Analysis

Intelligent Self-Healing Kubernetes Platform using AI-Driven Observability and Automated Remediation

---

A note on sourcing

Four of the five sources in the original reading list — the IEEE, ACM, and Springer papers — could not be located under their stated identifiers after individual searches for each. The Google SRE Book is genuine. For the other four, verify them yourself through IEEE Xplore / ACM Digital Library / Springer Link (MSRIT library login) before citing them anywhere in your report. The survey below is built entirely on sources I could independently confirm.

---

1. Foundational reliability engineering

Google's *Site Reliability Engineering* book is the standard reference for the vocabulary the rest of your report will lean on: SLIs/SLOs, error budgets, toil reduction, and treating reliability as an engineering discipline rather than firefighting. It's freely available at sre.google and is worth citing directly.

2. Kubernetes-native self-healing — the baseline you're improving on

Kubernetes already self-heals for a narrow set of failure modes: kubelet restarts a failed container per its restart policy, the ReplicaSet/Deployment/StatefulSet controllers replace missing Pods to match the desired count, and liveness/readiness probes pull unhealthy Pods out of Service endpoints [1]. This is the precise, checkable version of "repairs only predefined failures" — naming it specifically will read much better than gesturing at "current monitoring systems" in the abstract.

3. Detect-only systems: classical ML and LLM-agent anomaly detection

Matsuo and Ikegami (CNSM 2021) are the closest real-world analogue to your exact setup: a three-tier app (Nginx/Rails/MySQL) on Kubernetes with Prometheus, tested against HPA and native self-healing under injected load (LOCUST) and pod-kill faults (PUMBA) [2]. Two findings are directly usable in your gap section: (1) LSTM was their best detector (average AUROC 0.964, beating OCSVM, AutoEncoder, and LOF), and (2) K8s's own auto-scaling and self-healing don't always recover the system in time — response time took anywhere from roughly 1 to 9.5 minutes to start recovering depending on fault type, and the paper never proposes any remediation beyond what K8s already does. It's a measurement of the ceiling, not a system that raises it.

CloudAnoBench/CloudAnoAgent (2025) is the strongest classical-vs-LLM detection comparison available [4]: an LLM agent running a fast metrics pass and a slower log-reasoning pass, checked by a rule-based symbolic verifier, reaches 91.3% F1 / 12.9% false-positive rate jointly correlating metrics and logs — against roughly 54% F1 / 72% FPR for classical ML (Isolation Forest, K-Means) and roughly 73% F1 / 63% FPR for a bare LLM with no agent structure. Two caveats worth citing precisely: the symbolic verifier is scenario-specific (it needs a predefined library of anomaly types to check against), and scenario-identification accuracy drops sharply as the scenario set grows — 100% at 5 scenario types down to 78.6% at 29, because more categories means more semantic overlap between them. Neither this system nor Matsuo and Ikegami's takes any remediation action — both stop at diagnosis.

Yazdanparast's 2024 survey of self-healing software systems [3] confirms the shape of the gap from the other direction: across the broader (non-K8s-specific) self-healing literature, most work targets general self-adaptation rather than healing specifically, failure prediction and healing are studied as separate problems in most cited work, and — published in early 2024 — it doesn't cover the LLM-agent wave that arrived immediately after.

4. Autonomous, closed-loop remediation — what exists, and what it costs

This is the part of the original list most worth refreshing; AI-for-ops has moved fast in the last two to three years, and this tier is where the closest competitors to your actual proposal sit.

- **K8sGPT**, a CNCF sandbox project, scans a live cluster and turns raw diagnostic data into plain-language explanations through a pluggable AI backend, with rule-based auto-remediation layered on top of diagnosis [5]. Closest open-source relative, but user-triggered rather than continuously watching.
- **KubeIntellect** (2025) gives an LLM natural-language access to the *entire* Kubernetes API surface — read, write, delete, exec, RBAC, lifecycle — with dynamic tool synthesis for anything not pre-built (93% synthesis success across 200 test queries) [7]. Comprehensive as a copilot, but it only acts when a human types a request; nothing in the architecture watches metrics and decides on its own to intervene, and the evaluation never measures MTTR, MTBF, or anything tied to an actual incident.
- **Wang et al. (2025)** is architecturally the closest match to your whole pipeline: OpenTelemetry → feature extraction (autoencoder on metrics, Drain parsing + embeddings on logs, a GNN on trace-derived service graphs) → a domain-fine-tuned LLM (Llama-2-13B via supervised fine-tuning + DPO on 5,000 expert-labeled incidents) → remediation with policy checks, dry-run validation, and confidence-gated human-in-the-loop escalation [6]. Reported results are strong — 84.2% MTTR reduction, 95% F1, 91% automation — worth citing as an upper bound, but note it's self-reported from a small, new venue, and needs a curated, expert-labeled 5,000-case dataset plus GPU nodes to fine-tune — out of reach for a solo academic project.
- **STRATUS (NeurIPS 2025)** is the most rigorous evidence in the whole reading list for how hard full autonomy still is [8]. On two SRE benchmark suites (AIOpsLab, ITBench), even its best configuration (GPT-4o) mitigates only 50–69% of failure problems, taking roughly 14–29 minutes and up to $9 in LLM calls per incident — and needs a full transactional undo/rollback mechanism (their "Transactional No-Regression" property) just to reach that: remove it, and success collapses to 15–23%, because a failed attempt left un-rolled-back makes the system harder to fix on the next try. The authors are also candid that several "solved" cases exploited a quirk of the benchmark's fault injector rather than durably fixing anything. This is the current frontier, and it's still slow, costly, and right about half the time on hard problems.

None of these four use a classical-ML pipeline like the one you're planning — which is genuinely useful for you: it supports a "lightweight, classical-ML alternative to the LLM-agent approach" framing instead of competing with LLM agents head-on.

5. Commercial reference point

Dynatrace's Davis AI engine combines predictive, causal, and generative AI to auto-generate Kubernetes remediation artifacts and explain root causes in natural language [9]. Worth one paragraph in your report as "what a fully-resourced commercial platform already does," specifically so you can draw an honest line between that and what a single-developer academic project can realistically deliver.

6. Fault injection — for your evaluation methodology, not your related work

Chaos Mesh and LitmusChaos are the standard CNCF-hosted tools for Kubernetes chaos engineering, letting you inject pod, network, and resource failures in a controlled, repeatable way [10][11]. Your MTTR/MTBF/availability numbers are only as credible as the fault injection method behind them — this belongs in your tool stack, not just your survey.

7. Two design implications from STRATUS

- **Add a rollback path.** None of your six objectives currently say what happens if a remediation action doesn't work or makes things worse. STRATUS's own ablation shows this isn't optional — removing their undo mechanism dropped success from 69% to 15–23%. You don't need a full transactional

subsystem: even "if health checks don't improve within N seconds, revert the last action and escalate" in Objective 5 closes a real hole reviewers who know this literature will probe.

- **Consider AIOpsLab instead of building your testbed entirely from scratch.** It's the harness STRATUS itself evaluates against — open source (github.com/microsoft/AIOpsLab), and it already wraps Prometheus + Chaos Mesh + a maintained library of injectable failure scenarios into one framework [12]. Adapting it for Objective 3 could save real implementation time and gives you a standard, comparable methodology instead of an ad hoc one.

Comparative table

Tool	Type	Detects	Predicts	Auto-remediates	Correlates metrics+logs+traces	Notes
K8s-native (probes, HPA, controllers)	OSS, built-in	Limited (liveness/readiness only)	No	Yes — restart/replace only	No	Doesn't always recover in time (Matsuo & Ikegami: 1–9.5 min delay)
CloudAnoAgent	Research (LLM agent)	Yes, 91.3% F1	No	No	Yes (metrics+logs)	Detection only; accuracy drops as scenario count grows
K8sGPT	OSS (CNCF sandbox)	Yes, cluster scan	No	Limited, rule-based	Partial	Closest open-source relative
KubeIntellect	Research (open source)	Yes, on request	No	Yes, full K8s API, on request	Partial	Human-triggered, not anomaly-triggered; no MTTR measured
Chaos Mesh / LitmusChaos / AIOpsLab	OSS	N/A — fault injection	N/A	N/A	N/A	For testing, not production remediation
Wang et al. (LLM RCA + remediation)	Research	Yes, 95% F1	No	Yes, with HITL escalation	Yes, all three	Needs a fine-tuned 13B model + curated 5K-case dataset + GPU
STRATUS	Research (open source)	Yes	No	Yes, autonomous	Partial	50–69% success, 14–29 min & up to $9/incident;

Tool	Type	Detects	Predicts	Auto-remediates	Correlates metrics+logs+traces	Notes
						needs full undo/retry
Dynatrace Davis AI	Commercial	Yes	Yes (causal + predictive)	Yes, generates remediation artifacts	Yes	What "fully resourced" looks like
Your proposed system	OSS, academic	Yes (ML on metrics)	Stated goal — scope precisely	Yes (restart/scale/rollback)	Yes, if you keep all three signals	Sub-second inference, no LLM in the loop — see below

Where the actual gap is

With six papers read in full rather than by abstract, the landscape sorts into three tiers by what each actually closes:

- **Detect, don't act** — Matsuo & Ikegami, CloudAnoAgent. Good-to-excellent detection; zero remediation.
- **Full control, but only on command** — KubeIntellect. Acts on the whole K8s API, but only when a human asks; never measures MTTR/MTBF.
- **Autonomous and closed-loop, but LLM-driven** — Wang et al., STRATUS. Genuinely close the loop, but pay for it in latency (minutes per incident), cost (dollars per incident in inference), and either a curated fine-tuning dataset or a full undo/retry subsystem to be reliable.

Nobody in this set reports a classical-ML-driven system — sub-second inference, no LLM in the remediation decision — evaluated with SRE-standard metrics (Availability, MTTR, MTBF, Detection Accuracy) against controlled, repeatable fault injection on a real Kubernetes cluster. That combination is your project, and it's the actual empty seat at this table — not "nobody has tried self-healing," but "nobody has tried it this cheaply, this fast, and measured it this rigorously."

Revisiting the original four gap claims against this evidence:

- *"Detects but doesn't repair"* — true, and now you can name exactly who: CloudAnoAgent and Matsuo & Ikegami specifically, rather than "existing work" in the abstract.
- *"Repairs only predefined failures"* — true of K8s-native, and Matsuo & Ikegami's own measurements (HPA/self-healing not always recovering the system in time) are your best citation for it.
- *"Cannot correlate metrics, logs, and traces"* — the weakest of the four now. CloudAnoAgent and Wang et al. both do this correlation. Narrow the claim: nobody does it *without an LLM in the loop*.
- *"Lacks predictive analysis"* — the strongest and most durable claim. Every system reviewed — CloudAnoAgent, KubeIntellect, Wang et al., STRATUS — reacts to anomalies already underway; none forecasts a failure before its onset. Keep the early-warning framing in Objective 4; it's a genuinely open gap, not an assumed one.

References & download links

Core text (essential, free): Beyer, B., Jones, C., Petoff, J., Murphy, N.R. (eds.) *Site Reliability Engineering: How Google Runs Production Systems.* O'Reilly, 2016. Free, official: https://sre.google/sre-book/table-of-contents/

Numbered (matches in-text citations above):

1. Kubernetes documentation — *Self-Healing*. Free: https://kubernetes.io/docs/concepts/architecture/self-healing
2. Matsuo, Y. & Ikegami, D. "Performance Analysis of Anomaly Detection Methods for Application System on Kubernetes with Auto-scaling and Self-healing." *CNSM 2021*. **Paywalled** — no open-access copy found; access via IEEE Xplore through MSRIT library, or DOI: https://doi.org/10.23919/cnsm52442.2021.9615544
3. Yazdanparast, Z. "A Survey on Self-healing Software System." arXiv:2403.00455, 2024. Free PDF: https://arxiv.org/pdf/2403.00455
4. "Towards Generalizable Context-aware Anomaly Detection: A Large-scale Benchmark in Cloud Environments." arXiv:2508.01844, 2025. Free PDF: https://arxiv.org/pdf/2508.01844
5. K8sGPT project (CNCF Sandbox). Free, open source: https://github.com/k8sgpt-ai/k8sgpt
6. Wang, C. et al. "Integrating Large Language Models with Cloud-Native Observability for Automated Root Cause Analysis and Remediation." *Proc. 2025 3rd Int'l Conf. on AI, Systems and Network Security*. **Paywalled** — via ACM DL through MSRIT library: https://dl.acm.org/doi/10.1145/3797161.3797213
7. Ardebili, M.S. & Bartolini, A. "KubeIntellect: A Modular LLM-Orchestrated Agent Framework for End-to-End Kubernetes Management." arXiv:2509.02449, 2025. Free PDF: https://arxiv.org/pdf/2509.02449
8. Chen, Y. et al. "STRATUS: A Multi-Agent System for Autonomous Reliability Engineering of Modern Clouds." *NeurIPS 2025* (preprint). Free PDF: https://arxiv.org/pdf/2506.02009
9. Dynatrace. "Dynatrace Advances AIOps with Preventive Operations." Feb 2025. Free: https://www.businesswire.com/news/home/20250204836622/en/Dynatrace-Advances-AIOps-with-Preventive-Operations
10. Chaos Mesh (CNCF). Free, open source: https://chaos-mesh.org
11. LitmusChaos (CNCF). Free, open source: https://litmuschaos.io
12. AIOpsLab (Microsoft Research). Free, open source: https://github.com/microsoft/AIOpsLab